Entornos de Desarrollo

Unidad 1. Actividad 1

S. Francella Rojas Castillo

Contenido

Propiedades 2
¿Qué es el Sistema de Control de Versiones? (VSC) 2
Ventajas del Control de Versiones 2
Plataformas, Características y Diferencias 3
Características y Diferencias 3
Aplicaciones de Cliente de control de versiones 5
Control de versiones integrados en los IDES (plugins) 6
Bibliografía 7

Propiedades

¿Qué es el Sistema de Control de Versiones? (VSC)

Los sistemas de control de versiones son herramientas que nos permiten llevar un registro de todos los cambios que hacemos en el código con el paso del tiempo. Cada vez que modificamos algo, el sistema guarda una especie de “foto” de los archivos en ese momento. De esta forma, siempre podemos volver atrás y recuperar una versión anterior si lo necesitamos.

Si no existiera este tipo de control, la alternativa sería ir guardando varias copias del proyecto en el ordenador, algo bastante arriesgado. Es muy fácil equivocarse, modificar el archivo equivocado o incluso perder trabajo importante. Los sistemas de control de versiones evitan ese problema, ya que gestionan todas las versiones por nosotros, pero al mismo tiempo nos muestran una sola versión con la que trabajar.

Ventajas del Control de Versiones

→ Creación de flujos de trabajo: Evitan el caos de los usuarios que usan su propio proceso de desarrollo, muchas veces con herramientas diferentes e incompatibles.

→ Trabajo con versiones: Cada versión guarda una descripción de los cambios y puede consultarse o restaurarse en cualquier momento, facilitando el trabajo en equipo y la evolución segura del proyecto.

→ Codificar en conjunto con otros: El control de versiones mantiene todo sincronizado y evita conflictos cuando varias personas trabajan a la vez, ayudando al equipo a organizar y resolver cambios sin perder nada.

→ Mantener un historial: El control de versiones guarda un historial con quién hizo cada cambio, cuándo y por qué, lo que permite volver atrás fácilmente y trabajar con seguridad en cualquier versión del código.

→ Automatización de tareas: El control de versiones automatiza tareas como pruebas, análisis de código o implementación, ahorrando tiempo y garantizando resultados consistentes.

Plataformas, Características y Diferencias

En el mundo de las plataformas de control de versiones, hoy en día existen muchas, sin embargo, en Decroly por ejemplo utilizamos Sourcetree, sin embargo, esta es la lista de las más populares.

→ **GitHub:** La más popular, enfocada en proyectos colaborativos y open source.
→ **GitLab:** Similar a GitHub, con funciones integradas de CI/CD.
→ **Bitbucket:** Muy usada en entornos empresariales, integra bien con Jira.
→ **SourceForge:** De las primeras en ofrecer hosting para proyectos open source.
→ **AWS CodeCommit:** Servicio privado de repositorios en la nube de Amazon.
→ **Azure Repos:** Repositorios de código de Microsoft dentro de Azure DevOps.
→ **Google Cloud Source Repositories:** Repositorios privados en la nube de Google.
→ **Apache Subversion (SVN):** Un clásico del control de versiones centralizado, todavía usado en algunos.

Características y Diferencias

Para este apartado, creo que merece la pena hacer un cuadro comparativo, que recoge las diferencias más básicas y estructurales.

No sin antes aclarar la nomenclatura utilizada en el cuadro:

- SVN (Subversión): Indica si soporta Apache Subversion (un sistema de control de versiones más antiguo.
- Git: Si la plataforma soporta repositorios de Git.
- HG (Mercurial): Indica si soporta Mercurial, otro sistema de control de versiones distribuido (menos popular que Git, pero aún usado en algunos proyectos).
- Codereview: Señala si la plataforma incluye funciones para revisar código colaborativamente.
- Issue Tracker: Indica si la plataforma tiene integrado un sistema de seguimiento de incidencias/tareas (bugs, mejoras, pendientes), útil para organizar el trabajo del equipo.

Cuatro comparativo de Características:

Plataforma	Características
GitHub	La más popular en proyectos open source. Gran comunidad, pull requests, GitHub Actions (CI/CD), integración con miles de apps.
GitLab	Plataforma completa DevOps. Repositorios, CI/CD integrado, instalación en la nube o en servidores propios. Open source en su versión Community.
Bitbucket	Integración nativa con Jira y Trello. Ideal para equipos empresariales. Repositorios privados gratuitos. Ofrece pipelines para CI/CD.
SourceForge	Pionera en hosting de proyectos open source. Incluye foros, descargas y seguimiento de incidencias. Más orientada a proyectos públicos.
AWS CodeCommit	Repos privados en la nube de Amazon. Se integra con IAM (gestión de usuarios) y CodePipeline (CI/CD). Escalable y seguro para empresas.
Azure Repos (DevOps)	Parte de Azure DevOps. Ofrece repos Git y también soporte para SVN/TFS. Integración con Visual Studio y ecosistema Microsoft.
Google Cloud Source Repositories	Repos privados en GCP. Se integra con Cloud Build y Cloud IAM. Ideal para proyectos ya basados en Google Cloud.
Apache Subversion (SVN)	Sistema centralizado. Permite control detallado de permisos. Menos flexible que Git, pero aún usado en empresas tradicionales.

Tabla 1Tabla hecha en Excel. Características

Cuatro comparativo de diferencias:

Plataforma	Git	SVN	HG (Mercurial)	Revisión de código	Gestor de incidencias
GitHub	✓	✗	✗	✓	✓
GitLab	✓	✗	✗	✓	✓
Bitbucket	✓	✗	✗	✓	✓
SourceForge	✓	✓	✓	✗	✓
AWS CodeCommit	✓	✗	✗	✓	✗
Azure Repos (DevOps)	✓	✓	✗	✓	✓
Google Cloud Source Repositories	✓	✗	✗	✓	✗
Apache Subversion (SVN)	✗	✓	✗	✗	✗

Tabla 2Tabla hecha en Excel. Diferencias

Aplicaciones de Cliente de control de versiones

Estas aplicaciones son las que usan los clientes (usuarios como nosotros) para conectarse y trabajar con las plataformas de control de versiones explicadas en el apartado anterior.

A continuación, muestro un cuadro con las 8 principales que encontré.

Aplicación	Descripción principal	Sistema que soporta
Sourcetree	Cliente gratuito de Atlassian. Permite gestionar repositorios Git y Mercurial con interfaz gráfica. Ideal para principiantes y empresas.	Git, Mercurial
GitHub Desktop	Aplicación oficial de GitHub. Facilita clonar, crear ramas y subir cambios sin usar comandos.	Git
GitKraken	Cliente visual multiplataforma con herramientas avanzadas (gráficos de ramas, fusiones, conflictos). Gratuito para uso individual.	Git
TortoiseSVN	Cliente gráfico para Subversion (SVN). Se integra en el explorador de archivos de Windows.	SVN
TortoiseGit	Versión de Tortoise adaptada a Git. Muestra opciones en el menú contextual de Windows.	Git
SmartGit	Cliente profesional para Git, SVN y Mercurial, con múltiples funciones de comparación y fusión.	Git, SVN, Mercurial
Git Extensions	Interfaz gráfica ligera para Git en Windows, con integración a Visual Studio.	Git
Tower	Cliente premium para macOS y Windows, centrado en flujos profesionales y equipos grandes.	Git

Tabla 3 Tabla hecha en Excel con principales Aplicaciones

Control de versiones integrados en los IDES (plugins)

Los entornos de desarrollo (IDEs) como Visual Studio Code o IntelliJ (entre otros, pero estos son los más usados en Decroly, por eso los enfatizo como ejemplo) son las herramientas principales donde los programadores escribimos y gestionamos el código. Dentro de ellos se pueden instalar plugins o extensiones que integran funciones adicionales, como el control de versiones (por ejemplo, Git).

De esta manera los plugins nos ayudan a poder hacer todo centralizado y automatizado en un solo lugar mientras estamos trabajando, a través de la terminal.

Plugin / Extensión	IDEs	Función principal / Descripción
GitLens	Visual Studio Code	Permite visualizar el historial de cambios, autores y comparaciones directamente en el editor.
Git Graph	Visual Studio Code	Muestra un gráfico visual de las ramas y commits, facilitando el seguimiento del proyecto.
GitHub Pull Requests and Issues	Visual Studio Code	Conecta con GitHub para gestionar ramas, pull requests y tareas sin salir del IDE.
EGit	Eclipse	Integra Git dentro de Eclipse para realizar commits, pushes, merges y gestión de ramas.
Subclipse	Eclipse	Plugin que permite trabajar con repositorios Subversion (SVN) desde Eclipse.
Git Integration Plugin	IntelliJ IDEA, PyCharm, WebStorm (JetBrains)	Permite clonar, versionar y sincronizar repos Git dentro del entorno JetBrains.
Mercurial Integration Plugin	IntelliJ IDEA (JetBrains)	Añade soporte para Mercurial, facilitando commits y revisiones desde el entorno.
Team Explorer	Visual Studio (Microsoft)	Gestiona proyectos Git o Azure Repos (TFS) con herramientas integradas para equipos.
Azure Repos / TFS Plugin	Visual Studio, Visual Studio Code	Permite conectar el IDE con repositorios en Azure DevOps y manejar ramas o cambios.
Versioning Plugin	NetBeans	Añade compatibilidad con Git, Mercurial y Subversion, mostrando el estado del código en tiempo real.
VCS (Version Control System)	Android Studio	Integra Git y SVN para controlar versiones de proyectos móviles sin usar comandos externos.

Tabla 4Tabla creada en Excel IDEs-Plugins

Bibliografía

- Microsoft Learn (2024). *¿Qué es el control de versiones?, tomado de:* https://learn.microsoft.com/es-es/devops/develop/git/what-is-version-control
- Rewind (2023). *Comparativa: GitHub vs Bitbucket vs GitLab, tomado de:* https://rewind.com/blog/github-vs-bitbucket-vs-gitlab-comparison/?utm_source
- Tower Blog (2023). *Comparativa de servicios de hosting Git, tomado de:* https://www.git-tower.com/blog/git-hosting-services-compared?utm_source
- Pandora FMS Blog (2024). *Control de versiones: qué es, para qué sirve y tipos, tomado de:* https://pandorafms.com/blog/es/control-de-versiones/?utm_term=&utm_campaign=CSL+-+ES+-+ESP+-+PMAX+-+ITSM&utm_source=adwords&utm_medium=ppc&hsa_acc=2267641433&hsa_cam=22503347443&hsa_grp=&hsa_ad=&hsa_src=x&hsa_tgt=&hsa_kw=&hsa_mt=&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=22499880446&gbraid=0AAAAAD9cRmhKL2Vgrdf7Zgaz68J_y3pXW&gclid=Cj0KCQjwrojHBhDdARIsAJdEJ_cj_XW23z_UwLmu_iUVsiwRL7FsljQ5fN_Cr4nwFk5RazwEEY1011caAtXHEALw_wcB
- Códigonautas (2023). *Los mejores IDE y editores de texto para programadores, tomado de:* https://codigonautas.com/mejores-ides-editores-texto-programadores/